

Event Pairs lib コード行別コメント

対象: event_pairs/lib 配下の Dart ファイル 10 個 / 1076 行

このPDFは、コード初心者が各行の役割を追えるように、空行や閉じ括弧も含めて1行ずつ説明しています。

コメントは学習用の説明です。実際のソースコードへコメントを挿入したものではありません。

生成日時: 2026-06-30 02:15

event_pairs/lib/0-login_display.dart

0-login_display.dart は 23 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	import 'package:event_pairs/1-tab_page.dart';	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
4	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
5	class LoginDisplay extends StatelessWidget {	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
6	const LoginDisplay({super.key});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する助けをします。
7	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
8	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
9	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
10	return Scaffold(	1画面分の基本レイアウトを返します。本文や上部バーを置くための土台です。
11	body: Center(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
12	child: FilledButton(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
13	onPressed: () {	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
14	Navigator.of(context).pushReplacement(	別の画面へ移動するための処理です。タップ後の画面遷移に使います。
15	MaterialPageRoute<void>(builder: (context) => const TabPage()),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
16	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
17	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
18	child: const Text('登録する'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
19	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
20	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
21	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
22	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
23	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/1-tab_page.dart

1-tab_page.dart は 31 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	import 'package:event_pairs/4-my_page.dart';	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
4	import 'package:event_pairs/2-recruitment_display.dart';	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
5	import 'package:event_pairs/3-1-search_display.dart';	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
6	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
7	class TabPage extends StatelessWidget {	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
8	const TabPage({super.key});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
9	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
10	static const _tabs = <Tab>[	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
11	Tab(text: '見つける'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
12	Tab(text: '募集する'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
13	Tab(text: 'マイページ'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
14	];	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
15	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
16	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
17	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
18	return DefaultTabController(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
19	length: _tabs.length,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
20	child: Scaffold(	画面の基本構造を作る Flutter の部品です。上部バーや本文などを配置できます。
21	appBar: AppBar(	画面上部のバーを作ります。ページ名や戻る操作を分かりやすくします。
22	title: const Text('Event Pairs'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
23	bottom: const TabBar(tabs: _tabs),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
24	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
25	body: const TabBarView(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
26	children: <Widget>[SearchDisplay(), RecruitmentDisplay(), MyPage()],	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
27	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

28	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
29	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
30	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
31	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/2-recruitment_display.dart

2-recruitment_display.dart は 324 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/cupertino.dart';	iOS 風の部品を使えるようにします。時間選択の CupertinoDatePicker が必要です。
2	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
3	import 'package:event_pairs/event_store.dart';	イベントの保存・取得をまとめた自作ファイルを読み込みます。画面でイベント情報を使うためです。
4	import 'package:event_pairs/ollama_message_service.dart';	Ollama ヘ募集文生成を依頼する自作サービスを読み込みます。AI生成ボタンで使います。
5	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
6	class RecruitmentDisplay extends StatefulWidget {	値が変わる画面を表すクラスです。入力内容や選択状態を持つため StatefulWidget にしています。
7	const RecruitmentDisplay({super.key});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
8	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
9	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
10	State<RecruitmentDisplay> createState() => _RecruitmentDisplayState();	StatefulWidget が持つ状態管理クラスを作ります。画面の値が変わるため、この行で State と結びます。
11	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
12	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
13	class _RecruitmentDisplayState extends State<RecruitmentDisplay> {	StatefulWidget の状態を管理するクラスです。画面の変化する値と処理をここに置きます。
14	String purpose = '話したい';	現在選ばれている目的を文字列で保存します。選択中のボタン表示を変えるために使います。
15	int memberCount = 2;	募集人数の現在値を保存します。プラス・マイナスボタンで変化する数字です。
16	int maxMemberCount = 5;	募集人数の上限を保存します。5人までに制限するために使います。
17	EventCategory selectedCategory = EventCategory.circleVisit;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
18	bool isGeneratingMessage = false;	AI生成中かどうかを保存します。ボタンを無効化したり、生成中表示を出すためです。
19	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
20	final messageController = TextEditingController();	入力欄を管理する変数を作っています。一度作った管理役を同じ画面内で使い続けます。
21	final titleController = TextEditingController();	入力欄を管理する変数を作っています。一度作った管理役を同じ画面内で使い続けます。
22	final placeController = TextEditingController();	入力欄を管理する変数を作っています。一度作った管理役を同じ画面内で使い続けます。
23	final meetingTimeController = TextEditingController(text: "");	入力欄を管理する変数を作っています。一度作った管理役を同じ画面内で使い続けます。
24	final ollamaMessageService = OllamaMessageService();	AI募集文生成サービスを作っています。ボタンから同じサービスを呼び出せるようにします。
25	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
26	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
27	void dispose() {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。

28	<code>messageController.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
29	<code>titleController.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
30	<code>placeController.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
31	<code>meetingTimeController.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
32	<code>ollamaMessageService.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
33	<code>super.dispose();</code>	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
34	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
35	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
36	<code>void selectTime() {</code>	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
37	<code>showCupertinoModalPopup<void>(context: context, builder: (context) { return Container(height: 250, color: Colors.white, child: CupertinoDatePicker(mode: CupertinoDatePickerMode.time, use24hFormat: true, onDateTimeChanged: (time) { setState() { meetingTimeController.text = `\${time.hour.toString().padLeft(2, '0')}`; `\${time.minute.toString().padLeft(2, '0')}`; }); },),); },);</code>	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
38		設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
39		ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
40		この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
41		設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
42		設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
43		この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
44		設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
45		設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
46		ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
47		状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
48		この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
49		この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
50		Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
51		直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
52		ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
53		ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
54		直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
55		ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
56		直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。

57	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
58	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
59	void postEvent() {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
60	final title = titleController.text.trim();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
61	final place = placeController.text.trim();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
62	final message = messageController.text.trim();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
63	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
64	if (title.isEmpty) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
65	ScaffoldMessenger.of(	画面下に一時的な通知を出すための入口です。入力ミスや完了をユーザーへ伝えます。
66	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
67	).showSnackBar(const SnackBar(content: Text('タイトルを入力してください')));	短い通知バーを表示します。処理の結果をユーザーに分かりやすく知らせます。
68	return;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
69	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
70	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
71	EventStore.addEvent(	作成した募集イベントをアプリ内の保存場所へ追加します。一覧に反映するための中心になる処理です。
72	category: selectedCategory,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
73	title: title,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
74	description: message.isEmpty ? '一緒に行ける人を募集しています' : message,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
75	place: place.isEmpty ? '未定' : place,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
76	time: meetingTimeController.text,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
77	capacity: memberCount,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
78	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
79	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
80	titleController.clear();	入力欄の文字を空にします。投稿後に前の入力が残らないようにするためです。
81	placeController.clear();	入力欄の文字を空にします。投稿後に前の入力が残らないようにするためです。
82	messageController.clear();	入力欄の文字を空にします。投稿後に前の入力が残らないようにするためです。
83	setState(() {	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
84	purpose = '話したい';	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
85	memberCount = 2;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
86	selectedCategory = EventCategory.circleVisit;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
87	meetingTimeController.text = "";	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。

88	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
89	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
90	ScaffoldMessenger.of(	画面下に一時的な通知を出すための入口です。入力ミスや完了をユーザーへ伝えます。
91	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
92	).showSnackBar(const SnackBar(content: Text('イベントを作成しました')));	短い通知バーを表示します。処理の結果をユーザーに分かりやすく知らせます。
93	DefaultTabController.maybeOf(context)?.animateTo(0);	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
94	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
95	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
96	Future<void> generateMessage() async {	時間のかかる非同期処理を定義しています。通信や生成待ちを画面を止めずに扱うために必要です。
97	final title = titleController.text.trim();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
98	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
99	if (title.isEmpty) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
100	ScaffoldMessenger.of(	画面下に一時的な通知を出すための入口です。入力ミスや完了をユーザーへ伝えます。
101	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
102	).showSnackBar(const SnackBar(content: Text('先にタイトルを入力してください')));	短い通知バーを表示します。処理の結果をユーザーに分かりやすく知らせます。
103	return;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
104	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
105	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
106	setState(() {	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
107	isGeneratingMessage = true;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
108	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
109	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
110	try {	失敗する可能性がある処理をここから試します。通信エラーなどを安全に扱う準備です。
111	final message = await ollamaMessageService.generateRecruitmentMessage(	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
112	title: title,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
113	categoryLabel: selectedCategory.label,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
114	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
115	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
116	if (!mounted) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
117	return;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
118	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

119	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
120	setState() {	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
121	messageController.text = message;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
122	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
123	} catch (_) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
124	if (!mounted) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
125	return;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
126	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
127	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
128	ScaffoldMessenger.of(	画面下に一時的な通知を出すための入口です。入力ミスや完了をユーザーへ伝えます。
129	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
130	).showSnackBar(const SnackBar(content: Text('Ollamaからメッセージを作成できませんでした')));	短い通知バーを表示します。処理の結果をユーザーに分かりやすく知らせます。
131	} finally {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
132	if (mounted) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
133	setState() {	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
134	isGeneratingMessage = false;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
135	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
136	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
137	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
138	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
139	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
140	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
141	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
142	return SafeArea(	安全な表示領域を使った画面を返します。端末の端に内容が重ならないようにします。
143	top: false,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
144	child: SingleChildScrollView(	画面に収まりきらない内容をスクロールできるようにします。フォームや一覧表示に必要です。
145	padding: const EdgeInsets.all(16),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
146	child: Column(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
147	crossAxisAlignment: CrossAxisAlignment.start,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
148	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
149	const Text('どんな人と行きたい?', style: _titleStyle),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。

150	<code>const SizedBox(height: 12),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
151	<code>Row(</code>	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
152	<code> mainAxisAlignment: MainAxisAlignment.spaceBetween,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
153	<code> children: [</code>	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
154	<code> _purposeButton('話したい', Icons.chat_bubble_outline),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
155	<code> _purposeButton('仲間募集', Icons.groups_outlined),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
156	<code> _purposeButton('一緒に見学', Icons.location_on_outlined),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
157	<code> _purposeButton('その他', Icons.card_giftcard),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
158	<code>],</code>	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
159	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
160	<code>const SizedBox(height: 16),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
161	<code>const Text('タイトル', style: _titleStyle),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
162	<code>const SizedBox(height: 8),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
163	<code>TextField(</code>	ユーザーが文字を入力する欄を作ります。タイトル、場所、メッセージなどを受け取ります。
164	<code> controller: titleController,</code>	入力欄と <code>TextEditingController</code> を結びます。入力された文字をコード側で扱えるようにします。
165	<code> decoration: InputDecoration(</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
166	<code> filled: true,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
167	<code> fillColor: Colors.white,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
168	<code> border: OutlineInputBorder(</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
169	<code> borderRadius: BorderRadius.circular(14),</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
170	<code> borderSide: BorderSide.none,</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
171	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
172	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
173	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
174	<code>const SizedBox(height: 12),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
175	<code>const Text('カテゴリ', style: _titleStyle),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
176	<code>const SizedBox(height: 8),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
177	<code>_card(</code>	内容をまとめた枠として見せます。入力欄や情報を視覚的にグループ化します。
178	<code> child: DropdownButtonFormField<EventCategory>(</code>	複数の選択肢から1つを選ぶ部品です。イベントカテゴリの選択に使います。
179	<code> key: ValueKey(selectedCategory),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
180	<code> initialValue: selectedCategory,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

181	decoration: const InputDecoration(border: InputBorder.none),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
182	items: EventCategory.values.map((category) {	リストの各要素を別の形へ変換します。カテゴリからメニュー項目を作る時に使います。
183	return DropdownMenuItem<EventCategory>(value: category,	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
184	child: Text(category.label),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
185	);	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
186	)).toList(),	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
187	onChanged: (category) {	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
188	if (category == null) {	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
189	return;	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
190	}	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
191	(空行)	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
192	setState() {	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
193	selectedCategory = category;	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
194	});	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
195	},	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
196	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
197	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
198	const SizedBox(height: 12),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
199	const Text('場所', style: _titleStyle),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
200	const SizedBox(height: 8),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
201	TextField(controller: placeController,	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
202	decoration: InputDecoration(hintText: '例: 記念会館',	ユーザーが文字を入力する欄を作ります。タイトル、場所、メッセージなどを受け取ります。
203	filled: true,	入力欄と TextEditingController を結びます。入力された文字をコード側で扱うようにします。
204	fillColor: Colors.white,	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
205	border: OutlineInputBorder(borderRadius: BorderRadius.circular(14),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
206	borderSide: BorderSide.none,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
207	),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
208		色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
209		色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
210		色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
211		ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

212	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
213	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
214	const SizedBox(height: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
215	const Text('時間を選択', style: _titleStyle),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
216	_card(	内容をまとめた枠として見せます。入力欄や情報を視覚的にグループ化します。
217	child: ListTile(	タイトル、アイコン、タップ処理をまとめて表示できる行部品です。設定項目のような見た目に使います。
218	title: Text(meetingTimeController.text),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
219	trailing: const Icon(Icons.chevron_right),	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
220	onTap: selectTime,	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
221	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
222	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
223	const SizedBox(height: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
224	_card(	内容をまとめた枠として見せます。入力欄や情報を視覚的にグループ化します。
225	child: Row(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
226	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
227	const Icon(Icons.people_outline, color: Colors.orange),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
228	const SizedBox(width: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
229	const Expanded(child: Text('募集人数')),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
230	IconButton(	ユーザーが押せるボタンを作ります。投稿、生成、人数変更などの操作につなげます。
231	onPressed: memberCount > 1	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
232	? () => setState(() => memberCount--)	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
233	: null,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
234	icon: const Icon(Icons.remove_circle_outline),	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
235	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
236	Text('\$memberCount人まで'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
237	IconButton(	ユーザーが押せるボタンを作ります。投稿、生成、人数変更などの操作につなげます。
238	onPressed: (memberCount < maxMemberCount)	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
239	? () => setState(() => memberCount++)	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
240	: null,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

241	icon: const Icon(Icons.add_circle_outline),	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
242	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
243	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
244	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
245	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
246	const SizedBox(height: 24),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
247	Row(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
248	children: [	この部品の中に表示する子部品を指定します。Flutterの画面は親子構造で作られます。
249	const Expanded(child: Text('コメント (任意)', style: _titleStyle)),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
250	IconButton.icon(	ユーザーが押せるボタンを作ります。投稿、生成、人数変更などの操作につなげます。
251	onPressed: isGeneratingMessage ? null : generateMessage,	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
252	icon: isGeneratingMessage	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
253	? const SizedBox(	決まった余白やサイズを作ります。部品同士の間隔やボタンの高さを整えます。
254	width: 16,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
255	height: 16,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
256	child: CircularProgressIndicator(strokeWidth: 2),	この部品の中に表示する子部品を指定します。Flutterの画面は親子構造で作られます。
257	)	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
258	: const Icon(Icons.auto_awesome),	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
259	label: Text(isGeneratingMessage ? '生成中' : 'AIで作成'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
260	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
261	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
262	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
263	const SizedBox(height: 8),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
264	TextField(	ユーザーが文字を入力する欄を作ります。タイトル、場所、メッセージなどを受け取ります。
265	controller: messageController,	入力欄と TextEditingController を結びます。入力された文字をコード側で扱えるようにします。
266	maxLines: 5,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
267	decoration: InputDecoration(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
268	filled: true,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
269	fillColor: Colors.white,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

270	border: OutlineInputBorder(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
271	borderRadius: BorderRadius.circular(14),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
272	borderSide: BorderSide.none,	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
273	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
274	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
275	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
276	const SizedBox(height: 32),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
277	SizedBox(	決まった余白やサイズを作ります。部品同士の間隔やボタンの高さを整えます。
278	width: double.infinity,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
279	height: 52,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
280	child: ElevatedButton(	ユーザーが押せるボタンを作ります。投稿、生成、人数変更などの操作につなげます。
281	onPressed: postEvent,	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
282	child: const Text('募集を投稿する'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
283	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
284	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
285	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
286	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
287	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
288	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dartの文末として必要です。
289	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
290	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
291	Widget _purposeButton(String label, IconData icon) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
292	final selected = purpose == label;	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
293	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
294	return GestureDetector(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
295	onTap: () => setState(() => purpose = label),	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
296	child: Column(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
297	children: [	この部品の中に表示する子部品を指定します。Flutterの画面は親子構造で作られます。
298	CircleAvatar(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
299	backgroundColor: selected ? Colors.deepOrange : Colors.white,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
300	child: Icon(	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。

301	icon,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
302	color: selected ? Colors.white : Colors.deepOrange,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
303	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
304	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
305	const SizedBox(height: 6),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
306	Text(label, style: const TextStyle(fontSize: 12)),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
307	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
308	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
309	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
310	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
311	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
312	Widget _card({required Widget child}) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かず再利用できます。
313	return Material(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
314	color: Colors.white,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
315	shape: RoundedRectangleBorder(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
316	borderRadius: BorderRadius.circular(16),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
317	side: const BorderSide(color: Color(0xffe5e5e5)),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
318	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
319	child: Padding(padding: const EdgeInsets.all(14), child: child),	部品の内側や外側に余白を作ります。見た目を詰まりすぎないように整えます。
320	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
321	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
322	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
323	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
324	const _titleStyle = TextStyle(fontSize: 14, fontWeight: FontWeight.bold);	何度も使う文字スタイルを定数としてまとめています。見出しの見た目を統一できます。

event_pairs/lib/3-1-search_display.dart

3-1-search_display.dart は 131 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	<code>import 'package:flutter/material.dart';</code>	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	<code>import 'package:event_pairs/3-2-search_description.dart';</code>	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
3	<code>import 'package:event_pairs/event_store.dart';</code>	イベントの保存・取得をまとめた自作ファイルを読み込みます。画面でイベント情報を使うためです。
4	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
5	<code>class SearchDisplay extends StatelessWidget {</code>	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
6	<code>const SearchDisplay({super.key});</code>	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
7	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
8	<code>@override</code>	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
9	<code>Widget build(BuildContext context) {</code>	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
10	<code>return SafeArea(</code>	安全な表示領域を使った画面を返します。端末の端に内容が重ならないようにします。
11	<code>top: false,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
12	<code>child: ListView(</code>	画面に収まりきらない内容をスクロールできるようにします。フォームや一覧表示に必要です。
13	<code>children: [</code>	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
14	<code>const Padding(</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
15	<code>padding: EdgeInsets.fromLTRB(16, 16, 16, 8),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
16	<code>child: Text(</code>	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
17	<code>'イベント検索',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
18	<code>style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
19	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
20	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
21	<code>_eventCard(</code>	内容をまとめた枠として見せます。入力欄や情報を視覚的にグループ化します。
22	<code>context,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
23	<code>color: Color(0xFF2F93DD),</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
24	<code>title: 'サークル見学\nまとめ',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
25	<code>subtitle: 'いろんなサークルを\n見に行こう!',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
26	<code>imagePath: 'assets/images/circle_visit.png',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
27	<code>category: EventCategory.circleVisit,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

28	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
29	_eventCard(	内容をまとめた枠として見せます。入力欄や情報を視覚的にグルーブ化します。
30	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
31	color: Color(0xFF47B22),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
32	title: '履修登録相談\nまとめ',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
33	subtitle: '戦費や同級生に\n悩みを打ち明けよう!',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
34	imagePath: 'assets/images/open_seminar.png',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
35	category: EventCategory.openSeminar,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
36	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
37	_eventCard(	内容をまとめた枠として見せます。入力欄や情報を視覚的にグルーブ化します。
38	context,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
39	color: Color(0xFF48B98E),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
40	title: '飲食店募集\nまとめ',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
41	subtitle: '気になった飲食店に\n行こう!',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
42	imagePath: 'assets/images/festival_staff.png',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
43	category: EventCategory.foodRecruitment,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
44	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
45	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
46	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
47	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
48	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
49	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
50	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
51	Widget _eventCard(	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
52	BuildContext context, {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
53	required Color color,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
54	required String title,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
55	required String subtitle,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
56	required String imagePath,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
57	required EventCategory category,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
58	)) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。

59	return Card(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
60	clipBehavior: Clip.antiAlias,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
61	elevation: 3,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
62	shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
63	child: InkWell(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
64	onTap: () {	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力動きにつなげます。
65	Navigator.of(context).push(	別の画面へ移動するための処理です。タップ後の画面遷移に使います。
66	MaterialPageRoute<void>(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
67	builder: (context) => EventListPage(category: category),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
68	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
69	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
70	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
71	child: SummaryCardContent(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
72	color: color,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
73	title: title,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
74	subtitle: subtitle,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
75	imagePath: imagePath,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
76	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
77	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
78	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
79	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
80	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
81	class SummaryCardContent extends StatelessWidget {	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
82	final Color color;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
83	final String title;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
84	final String subtitle;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
85	final String imagePath;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
86	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
87	const SummaryCardContent({	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
88	super.key,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
89	required this.color,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。

90	required this.title,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
91	required this.subtitle,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
92	required this.imagePath,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
93	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
94	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
95	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
96	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
97	return Container(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
98	decoration: BoxDecoration(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
99	color: color,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
100	borderRadius: BorderRadius.circular(14),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
101	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
102	padding: const EdgeInsets.all(16),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
103	child: Row(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
104	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
105	Expanded(	空いている幅や高さを埋めるように部品を広げます。レイアウトの崩れを防ぐために使います。
106	child: Column(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
107	crossAxisAlignment: CrossAxisAlignment.start,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
108	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
109	Text(	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
110	title,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
111	style: const TextStyle(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
112	fontSize: 18,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
113	fontWeight: FontWeight.bold,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
114	color: Colors.white,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
115	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
116	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
117	const SizedBox(height: 8),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
118	Text(	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
119	subtitle,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
120	style: const TextStyle(fontSize: 14, color: Colors.white70),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

121	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
122	],	複数の子部品やリスト要素の並びをここで閉じます。親部品ヘリストを渡すために必要です。
123	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
124	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
125	const SizedBox(width: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
126	Image.asset(imagePath, width: 60, height: 80, fit: BoxFit.cover),	アプリ内に登録した画像ファイルを表示します。カテゴリカードを視覚的に分かりやすくします。
127	],	複数の子部品やリスト要素の並びをここで閉じます。親部品ヘリストを渡すために必要です。
128	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
129	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
130	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
131	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/3-2-search_description.dart

3-2-search_description.dart は 90 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	import 'package:event_pairs/event_store.dart';	イベントの保存・取得をまとめた自作ファイルを読み込みます。画面でイベント情報を使うためです。
4	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
5	class EventListPage extends StatelessWidget {	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
6	final EventCategory category;	イベントカテゴリを保存する変数です。イベントがどのジャンルに分かるようにします。
7	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
8	const EventListPage({super.key, required this.category});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
9	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
10	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
11	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
12	return Scaffold(	1画面分の基本レイアウトを返します。本文や上部バーを置くための土台です。
13	appBar: AppBar(title: Text('\${category.label} 一覧')),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
14	body: ValueListenableBuilder<int>{(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
15	valueListenable: EventStore.version,	イベント情報をまとめて管理する EventStore を使っています。画面同士で同じデータを共有できます。
16	builder: (context, __, __) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
17	final events = EventStore.eventsByCategory(category);	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
18	return ListView.separated(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
19	padding: const EdgeInsets.all(16),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
20	itemCount: events.length,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
21	separatorBuilder: (context, index) => const SizedBox(height: 12),	決まった余白やサイズを作ります。部品同士の間隔やボタンの高さを整えます。
22	itemBuilder: (context, index) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
23	final event = events[index];	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
24	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
25	return Card(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
26	child: ListTile(	タイトル、アイコン、タップ処理をまとめて表示できる行部品です。設定項目のような見た目に使います。
27	title: Text(event.title),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。

28	subtitle: Text('\${event.time} / \${event.place}'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
29	trailing: const Icon(Icons.chevron_right),	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
30	onTap: () => _showApplySheet(context, event),	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
31	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
32	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
33	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
34	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
35	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
36	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
37	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
38	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
39	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
40	void _showApplySheet(BuildContext context, EventInfo event) {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
41	final applied = EventStore.hasApplied(event.id);	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
42	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
43	showModalBottomSheet<void>(context: context, builder: (sheetContext) { return SafeArea(child: Padding(padding: const EdgeInsets.all(20), child: Column(mainAxisSize: MainAxisSize.min, crossAxisAlignment: CrossAxisAlignment.start, children: [Text(event.title, style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold,)),	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。 安全な表示領域を使った画面を返します。端末の端に内容が重ならないようにします。 部品の内側や外側に余白を作ります。見た目を詰まりすぎないように整えます。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。 画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

59	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
60	const SizedBox(height: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
61	Text(event.description),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
62	const SizedBox(height: 12),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
63	Text('場所: \${event.place}'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
64	Text('時間: \${event.time}'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
65	Text('募集人数: \${event.capacity}人まで'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
66	Text('作成者: \${event.ownerName}'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
67	const SizedBox(height: 20),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
68	SizedBox(	決まった余白やサイズを作ります。部品同士の間隔やボタンの高さを整えます。
69	width: double.infinity,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
70	child: FilledButton(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
71	onPressed: applied	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力動きにつなげます。
72	? null	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
73	: () {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
74	EventStore.apply(event);	選んだイベントへ申請済みとして登録します。マイページに表示するために必要です。
75	Navigator.pop(sheetContext);	別の画面へ移動するための処理です。タップ後の画面遷移に使います。
76	ScaffoldMessenger.of(context).showSnackBar(	画面下に一時的な通知を出すための入口です。入力ミスや完了をユーザーへ伝えます。
77	const SnackBar(content: Text('申請しました')),	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
78	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
79	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
80	child: Text(applied ? '申請済み' : '申請する'),	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
81	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
82	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
83	],	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
84	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
85	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
86	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
87	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
88	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。

89	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
90	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/4-my_page.dart

4-my_page.dart は 54 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	import 'package:event_pairs/5-message.dart';	メッセージ画面のファイルを読み込みます。マイページからチャット画面へ移動するためです。
4	import 'package:event_pairs/event_store.dart';	イベントの保存・取得をまとめた自作ファイルを読み込みます。画面でイベント情報を使うためです。
5	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
6	class MyPage extends StatelessWidget {	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
7	const MyPage({super.key});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
8	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
9	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
10	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かず再利用できます。
11	return SafeArea(	安全な表示領域を使った画面を返します。端末の端に内容が重ならないようにします。
12	top: false,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
13	child: ValueListenableBuilder<int>(valueListenable: EventStore.version, builder: (context, _, __) { final applications = EventStore.applications; (空行) return ListView.separated(padding: const EdgeInsets.all(16), itemCount: applications.length, separatorBuilder: (context, index) => const SizedBox(height: 12), itemBuilder: (context, index) { final application = applications[index]; final event = application.event; (空行) return Card(child: ListTile(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。 イベント情報をまとめて管理する EventStore を使っています。画面同士で同じデータを共有できます。 ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。 あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。 コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。 この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。 決まった余白やサイズを作ります。部品同士の間隔やボタンの高さを整えます。 ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。 あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。 あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。 コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。 この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。 タイトル、アイコン、タップ処理をまとめて表示できる行部品です。設定項目のような見た目に使います。
14		
15		
16		
17		
18		
19		
20		
21		
22		
23		
24		
25		
26		
27		

28	<code>title: Text(event.title),</code>	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
29	<code>subtitle: Text(</code>	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
30	<code>'\${event.category.label} / \${event.time} / \${event.place}',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
31	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
32	<code>leading: const Icon(Icons.check_circle_outline),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
33	<code>trailing: const Icon(Icons.chat_bubble_outline),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
34	<code>onTap: () {</code>	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力動きにつなげます。
35	<code>Navigator.of(context).push(</code>	別の画面へ移動するための処理です。タップ後の画面遷移に使います。
36	<code>MaterialPageRoute<void>(</code>	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
37	<code>builder: (context) {</code>	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
38	<code>return Scaffold(</code>	1画面分の基本レイアウトを返します。本文や上部バーを置くための土台です。
39	<code>appBar: AppBar(title: Text(event.title)),</code>	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
40	<code>body: const MessagePage(),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
41	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
42	<code>},</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
43	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
44	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
45	<code>},</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
46	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
47	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
48	<code>},</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
49	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
50	<code>},</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
51	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
52	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
53	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
54	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/5-message.dart

5-message.dart は 154 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/material.dart';	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	class MessagePage extends StatefulWidget {	値が変わる画面を表すクラスです。入力内容や選択状態を持つため StatefulWidget にしています。
4	const MessagePage({super.key});	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
5	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
6	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
7	State<MessagePage> createState() => _MessagePageState();	StatefulWidget が持つ状態管理クラスを作ります。画面の値が変わるため、この行で State と結びます。
8	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
9	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
10	class _MessagePageState extends State<MessagePage> {	StatefulWidget の状態を管理するクラスです。画面の変化する値と処理をここに置きます。
11	final messageController = TextEditingController();	入力欄を管理する変数を作っています。一度作った管理役を同じ画面内で使い続けます。
12	final scrollController = ScrollController();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
13	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
14	final List<ChatMessage> messages = [	複数のデータをまとめるリストです。イベント一覧や申請一覧を管理するために使います。
15	ChatMessage(text: 'こんにちは！イベントについて気軽に相談できます。', isMe: false),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
16	];	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
17	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
18	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
19	void dispose() {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
20	messageController.dispose();	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
21	scrollController.dispose();	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
22	super.dispose();	使い終わった管理オブジェクトを片付けます。無駄なメモリ使用を防ぐために必要です。
23	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
24	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
25	void sendMessage() {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
26	final text = messageController.text.trim();	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
27	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。

28	<code>if (text.isEmpty) {</code>	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
29	<code>return;</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
30	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
31	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
32	<code>setState(() {</code>	状態を変更したことを Flutter に知らせます。これがあると画面が新しい値で再描画されます。
33	<code>messages.add(ChatMessage(text: text, isMe: true));</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
34	<code>messageController.clear();</code>	入力欄の文字を空にします。投稿後に前の入力が残らないようにするためです。
35	<code>});</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
36	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
37	<code>scrollToBottom();</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
38	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
39	<code>Future.delayed(const Duration(milliseconds: 500), () {</code>	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
40	<code>if (!mounted) {</code>	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
41	<code>return;</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
42	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
43	<code>scrollToBottom();</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
44	<code>});</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
45	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
46	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
47	<code>void scrollToBottom() {</code>	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
48	<code>WidgetsBinding.instance.addPostFrameCallback((_) {</code>	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
49	<code>if (!scrollController.hasClients) {</code>	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
50	<code>return;</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
51	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
52	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
53	<code>scrollController.animateTo(</code>	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
54	<code>scrollController.position.maxScrollExtent,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
55	<code>duration: const Duration(milliseconds: 250),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
56	<code>curve: Curves.easeOut,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
57	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
58	<code>});</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。

59	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
60	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
61	@override	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
62	Widget build(BuildContext context) {	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
63	return SafeArea(	安全な表示領域を使った画面を返します。端末の端に内容が重ならないようにします。
64	top: false,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
65	child: Column(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
66	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
67	Expanded(	空いている幅や高さを埋めるように部品を広げます。レイアウトの崩れを防ぐために使います。
68	child: ListView.builder(	画面に収まりきらない内容をスクロールできるようにします。フォームや一覧表示に必要です。
69	controller: scrollController,	入力欄と TextEditingController を結びます。入力された文字をコード側で扱えるようにします。
70	padding: const EdgeInsets.all(16),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
71	itemCount: messages.length,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
72	itemBuilder: (context, index) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
73	final message = messages[index];	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
74	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
75	return Align(	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
76	alignment: message.isMe	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
77	? Alignment.centerRight	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
78	: Alignment.centerLeft,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
79	child: Container(	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
80	margin: const EdgeInsets.only(bottom: 10),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
81	padding: const EdgeInsets.symmetric(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
82	horizontal: 14,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
83	vertical: 10,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
84	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
85	constraints: BoxConstraints(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
86	maxWidth: MediaQuery.sizeOf(context).width * 0.72,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
87	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

88	decoration: BoxDecoration(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
89	color: message.isMe	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
90	? const Color(0xFF2F93DD)	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
91	: const Color(0xFFFF1F3F5),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
92	borderRadius: BorderRadius.circular(16),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
93	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
94	child: Text(	画面に文字を表示する部品です。見出しや説明文としてユーザーに情報を見せます。
95	message.text,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
96	style: TextStyle(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
97	color: message.isMe ? Colors.white : Colors.black87,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
98	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
99	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
100	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
101	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
102	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
103	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
104	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
105	Container(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
106	padding: const EdgeInsets.fromLTRB(12, 8, 12, 12),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
107	decoration: const BoxDecoration(	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
108	color: Colors.white,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
109	border: Border(top: BorderSide(color: Color(0xFFE5E5E5))),	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
110	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
111	child: Row(	複数の部品を縦または横に並べます。画面レイアウトの基本になる行です。
112	children: [	この部品の中に表示する子部品を指定します。Flutter の画面は親子構造で作られます。
113	Expanded(	空いている幅や高さを埋めるように部品を広げます。レイアウトの崩れを防ぐために使います。
114	child: TextField(	ユーザーが文字を入力する欄を作ります。タイトル、場所、メッセージなどを受け取ります。
115	controller: messageController,	入力欄と TextEditingController を結びます。入力された文字をコード側で扱えるようにします。
116	minLines: 1,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
117	maxLines: 4,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

118	<code>textInputAction: TextInputAction.send,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
119	<code>decoration: InputDecoration(</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
120	<code>hintText: 'メッセージを入力',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
121	<code>filled: true,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
122	<code>fillColor: const Color(0xFF7F7F7F),</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
123	<code>border: OutlineInputBorder(</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
124	<code>borderRadius: BorderRadius.circular(18),</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
125	<code>borderSide: BorderSide.none,</code>	色、角丸、枠線などの見た目を設定します。UIを読みやすく整えるための行です。
126	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
127	<code>contentPadding: const EdgeInsets.symmetric(</code>	部品の内側や外側に余白を作ります。見た目を詰まりすぎないように整えます。
128	<code>horizontal: 14,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
129	<code>vertical: 10,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
130	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
131	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
132	<code>onSubmitted: (_) => sendMessage(),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
133	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
134	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
135	<code>const SizedBox(width: 8),</code>	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
136	<code>IconButton.filled(</code>	ユーザーが押せるボタンを作ります。投稿、生成、人数変更などの操作につなげます。
137	<code>onPressed: sendMessage,</code>	ユーザー操作が起きた時に実行する処理を指定します。ボタンや入力を動きにつなげます。
138	<code>icon: const Icon(Icons.send),</code>	画面にアイコンを表示します。操作や意味を文字だけより直感的に伝えます。
139	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
140	<code>],</code>	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
141	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
142	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
143	<code>],</code>	複数の子部品やリスト要素の並びをここで閉じます。親部品へリストを渡すために必要です。
144	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
145	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dartの文末として必要です。
146	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
147	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
148	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。

149	class ChatMessage {	関連する変数や処理をまとめる設計図を定義しています。Dart ではクラス単位で機能を整理します。
150	final String text;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
151	final bool isMe;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
152	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
153	const ChatMessage({required this.text, required this.isMe});	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
154	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/event_store.dart

event_store.dart は 137 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'package:flutter/foundation.dart';	ValueNotifier など、Flutter の基礎機能を使うための読み込みです。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	enum EventCategory {	選べる値の種類を固定する定義です。カテゴリのように決まった候補だけを扱いたい時に使います。
4	circleVisit('サークル見学'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
5	openSeminar('履修登録相談'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
6	foodRecruitment('飲食店募集');	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
7	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
8	const EventCategory(this.label);	enum の各値に表示名を結び付けるためのコンストラクタです。カテゴリ名を画面に出せます。
9	final String label;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
10	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
11	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
12	class EventInfo {	イベント1件分のデータの形を定義します。タイトル、場所、時間などをまとめて扱えます。
13	final String id;	一度決めたIDを保存する文字列です。データを区別するために変更しない値として持ちます。
14	final EventCategory category;	イベントカテゴリを保存する変数です。イベントがどのジャンルに分かるようにします。
15	final String title;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
16	final String description;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
17	final String place;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
18	final String time;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
19	final int capacity;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
20	final String ownerName;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
21	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
22	const EventInfo({	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
23	required this.id,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
24	required this.category,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
25	required this.title,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
26	required this.description,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
27	required this.place,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。

28	required this.time,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
29	required this.capacity,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
30	required this.ownerName,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
31	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
32	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
33	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
34	class EventApplication {	申請1件分のデータの形を定義します。どのイベントにいつ申請したかをまとめます。
35	final String id;	一度決めたIDを保存する文字列です。データを区別するために変更しない値として持ちます。
36	final EventInfo event;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
37	final DateTime appliedAt;	日時を保存する変数です。申請した時刻などを記録できます。
38	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
39	const EventApplication({	コンパイル時に固定できる値や作り方を定義しています。変わらないものを効率よく扱えます。
40	required this.id,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
41	required this.event,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
42	required this.appliedAt,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
43	});	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
44	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
45	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
46	class EventStore {	イベントや申請をアプリ内で管理する置き場を定義します。複数画面で同じデータを使うためです。
47	static final version = ValueNotifier<int>(0);	クラスから直接使える共有データを作っています。アプリ全体で同じ情報を参照するために必要です。
48	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
49	static final List<EventInfo> _events = [	クラスから直接使える共有データを作っています。アプリ全体で同じ情報を参照するために必要です。
50	EventInfo(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
51	id: 'event_1',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
52	category: EventCategory.circleVisit,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
53	title: '軽音サークル見学',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
54	description: '初心者歓迎。昼休みに部室を見に行きます。',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
55	place: '学生会館 2F',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
56	time: '12:40',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
57	capacity: 4,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
58	ownerName: '田中',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

59	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
60	EventInfo(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
61	id: 'event_2',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
62	category: EventCategory.openSeminar,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
63	title: '履修登録と一緒に考える会',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
64	description: 'おすすめ授業や時間割の組み方を相談できます。',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
65	place: '図書館前',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
66	time: '14:00',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
67	capacity: 6,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
68	ownerName: '佐藤',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
69	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
70	EventInfo(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
71	id: 'event_3',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
72	category: EventCategory.foodRecruitment,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
73	title: '学食で昼ごはん',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
74	description: '空きコマと一緒にごはん行ける人募集。',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
75	place: '第一食堂',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
76	time: '12:15',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
77	capacity: 3,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
78	ownerName: '山田',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
79	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
80	];	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
81	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
82	static final List<EventApplication> _applications = [];	クラスから直接使える共有データを作っています。アプリ全体で同じ情報を参照するために必要です。
83	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
84	static List<EventInfo> eventsByCategory(EventCategory category) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
85	return _events.where((event) => event.category == category).toList();	指定カテゴリだけに絞ったイベント一覧を返します。カテゴリ別一覧表示に必要です。
86	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
87	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
88	static List<EventApplication> get applications {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
89	return List.unmodifiable(_applications);	外から直接変更できないリストとして返します。データ管理の安全性を高めます。

90	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
91	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
92	static bool hasApplied(String eventId) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
93	return _applications.any((application) => application.event.id == eventId);	同じイベントへ申請済みかどうかの判定結果を返します。重複申請を防ぎます。
94	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
95	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
96	static void addEvent({	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
97	required EventCategory category,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
98	required String title,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
99	required String description,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
100	required String place,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
101	required String time,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
102	required int capacity,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
103	)) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
104	final id = DateTime.now().microsecondsSinceEpoch.toString();	日時を保存する変数です。申請した時刻などを記録できます。
105	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
106	_events.insert(	リストの指定位置に新しいデータを追加します。新しいイベントを先頭に出すために使います。
107	0,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
108	EventInfo(	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
109	id: id,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
110	category: category,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
111	title: title,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
112	description: description,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
113	place: place,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
114	time: time,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
115	capacity: capacity,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
116	ownerName: 'あなた',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
117	),	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
118	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
119	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
120	version.value++;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。

121	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
122	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
123	static void apply(EventInfo event) {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
124	if (hasApplied(event.id)) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
125	return;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
126	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
127	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
128	final id = DateTime.now().microsecondsSinceEpoch.toString();	日時を保存する変数です。申請した時刻などを記録できます。
129	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
130	_applications.insert(	リストの指定位置に新しいデータを追加します。新しいイベントを先頭に出すために使います。
131	0,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
132	EventApplication(id: id, event: event, appliedAt: DateTime.now()),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
133	);	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
134	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
135	version.value++;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
136	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
137	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/main.dart

main.dart は 23 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	<code>import 'package:flutter/material.dart';</code>	Material Design の画面部品を使えるようにします。Text や Scaffold など多くの基本UIに必要です。
2	<code>import 'package:event_pairs/0-login_display.dart';</code>	別ファイルやライブラリの機能をこのファイルで使えるように読み込む行です。
3	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
4	<code>void main() {</code>	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
5	<code> runApp(const MyApp());</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
6	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
7	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
8	<code>class MyApp extends StatelessWidget {</code>	状態を持たない画面や部品を表すクラスです。受け取った情報を表示する時に使います。
9	<code> const MyApp({super.key});</code>	このウィジェットを作るためのコンストラクタです。super.key は Flutter が部品を識別する手助けをします。
10	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
11	<code> @override</code>	親クラスや Flutter が用意したメソッドを、このクラス用に上書きして使うことを示します。
12	<code> Widget build(BuildContext context) {</code>	画面部品を返す小さな関数を定義しています。同じ見た目を何度も書かずに再利用できます。
13	<code> return MaterialApp(</code>	Flutter アプリ全体の土台を返します。テーマや最初の画面をここから設定します。
14	<code> title: 'Event Pairs',</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
15	<code> debugShowCheckedModeBanner: false,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
16	<code> theme: ThemeData(</code>	ここから複数行に分けて設定を書き始めます。長い部品設定を読みやすくするためです。
17	<code> colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
18	<code> visualDensity: VisualDensity.adaptivePlatformDensity,</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
19	<code>),</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
20	<code> home: const LoginDisplay(),</code>	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
21	<code>);</code>	直前まで複数行で書いていた命令やウィジェット作成をここで完了します。Dart の文末として必要です。
22	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
23	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。

event_pairs/lib/ollama_message_service.dart

ollama_message_service.dart は 109 行です。下の表で、左から行番号、実際のコード、その行の役割を確認できます。

行	コード	初心者向けコメント
1	import 'dart:convert';	JSON の変換機能を使えるようにします。Ollama API とデータをやり取りするために必要です。
2	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
3	import 'package:http/http.dart' as http;	HTTP 通信ライブラリを使えるようにします。アプリから Ollama にリクエストを送るためです。
4	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
5	class OllamaMessageService {	Ollama との通信を担当するクラスです。画面側のコードを通信処理でごちゃつかせないためです。
6	OllamaMessageService({	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
7	http.Client? client,	HTTP 通信で Ollama のローカル API にリクエストを送ります。AI生成をアプリから呼ぶための行です。
8	this.baseUrl = 'http://127.0.0.1:11434',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
9	this.model = 'qwen3:4b',	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
10	}) : _client = client ?? http.Client();	HTTP 通信で Ollama のローカル API にリクエストを送ります。AI生成をアプリから呼ぶための行です。
11	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
12	final http.Client _client;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
13	final String baseUrl;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
14	final String model;	このクラスが持つ変更しない値を宣言しています。必要なデータを分かりやすく名前で持たせます。
15	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
16	void dispose() {	値を返さない処理のまとまりを定義しています。ボタン操作などで呼び出すために名前を付けています。
17	_client.close();	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
18	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
19	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
20	Future<String> generateRecruitmentMessage({	時間のかかる非同期処理を定義しています。通信や生成待ちを画面を止めずに扱うために必要です。
21	required String title,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
22	required String categoryLabel,	この値は必ず渡す必要があると示します。必要なデータの渡し忘れを防げます。
23	}) async {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
24	final assistantPrefix = '大学生のみなさん、';	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
25	final prompt =	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
26	'< im_start >system\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。

27	'あなたは面白おかしく大学生向けイベントの短い募集文を書くAIです。'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
28	'日本語で、説明や思考過程は出さず、面白い1文だけを書いてください。'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
29	'< im_end >\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
30	'< im_start >user\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
31	'タイトル: \$title\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
32	'カテゴリ: \$categoryLabel\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
33	'募集文を面白おかしく1文で作成してください。'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
34	'< im_end >\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
35	'< im_start >assistant\n'	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
36	'\$assistantPrefix';	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
37	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
38	final response = await _client	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
39	.post(	HTTP 通信で Ollama のローカル API にリクエストを送ります。AI生成をアプリから呼ぶための行です。
40	Uri.parse('\$baseUrl/api/generate'),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
41	headers: const {'Content-Type': 'application/json'},	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
42	body: jsonEncode({	Dart の Map を JSON 文字列に変換します。Ollama API にデータを送るために必要です。
43	'model': model,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
44	'stream': false,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
45	'raw': true,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
46	'think': false,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
47	'prompt': prompt,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
48	'options': {	ここから新しい処理や設定のまとまりが始まります。中に書いた行がこのブロックに属します。
49	'num_predict': 80,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
50	'temperature': 0.2,	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
51	'stop': ['.', '!', '?', '< im_end >', '< im_start >'],	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。

52	},	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
53	)),	設定値や要素を1つ追加しています。末尾のカンマで次の設定を続けやすくしています。
54	)	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
55	.timeout(const Duration(seconds: 15));	通信が長く止まりすぎないように制限時間を付けます。アプリが待ちっぱなしになるのを防ぎます。
56	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
57	if (response.statusCode != 200) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
58	throw Exception('Ollama returned \${response.statusCode}');	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
59	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
60	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
61	final json = jsonDecode(utf8.decode(response.bodyBytes));	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
62	final generatedText = json is Map<String, dynamic>	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
63	? json['response'] as String? ?? "	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
64	:";	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
65	final cleanedText = _cleanGeneratedText(generatedText);	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
66	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
67	if (cleanedText.isEmpty) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
68	throw Exception('Ollama returned an empty message');	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
69	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
70	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
71	return _completeSentence(assistantPrefix, cleanedText);	整えた文字列を呼び出し元へ返します。画面のメッセージ欄に入れるためです。
72	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
73	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
74	String _cleanGeneratedText(String text) {	変化する値を保存する変数です。画面の状態や処理に必要な情報を名前付きで持ちます。
75	var cleaned = text;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
76	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
77	if (cleaned.contains('</think>')) {	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
78	cleaned = cleaned.split('</think>').last;	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
79	}	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
80	(空行)	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。

81	<code>cleaned = cleaned.replaceAll(RegExp(r'<think>[\s\S]*'), "");</code>	不要な文字やタグを取り除いて、画面に出しやすい文章へ整えます。AIの余分な出力対策です。
82	<code>cleaned = cleaned.replaceAll(RegExp(r'<^>+>'), "");</code>	不要な文字やタグを取り除いて、画面に出しやすい文章へ整えます。AIの余分な出力対策です。
83	<code>cleaned = cleaned.replaceAll("\n", ' ');</code>	不要な文字やタグを取り除いて、画面に出しやすい文章へ整えます。AIの余分な出力対策です。
84	<code>cleaned = cleaned.replaceAll(RegExp(r'\s+'), ' ');</code>	不要な文字やタグを取り除いて、画面に出しやすい文章へ整えます。AIの余分な出力対策です。
85	<code>cleaned = cleaned.trim();</code>	文字列の前後の空白を取り除きます。空白だけの入力を正しく空として扱えます。
86	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
87	<code>if (cleaned.startsWith('「') && cleaned.endsWith('」')) {</code>	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
88	<code>cleaned = cleaned.substring(1, cleaned.length - 1).trim();</code>	文字列の前後の空白を取り除きます。空白だけの入力を正しく空として扱えます。
89	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
90	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
91	<code>return cleaned;</code>	整えた文字列を呼び出し元へ返します。画面のメッセージ欄に入れるためです。
92	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
93	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
94	<code>String _completeSentence(String prefix, String generatedText) {</code>	変化する値を保存する変数です。画面の状態や処理に必要な情報を名前付きで持ちます。
95	<code>final message = generatedText.startsWith(prefix)</code>	あとから別のものに入れ替えない値を作っています。安全に同じ参照を使い続けるためです。
96	<code>? generatedText</code>	この行は前後の行とセットで、値の指定や処理の一部を担当しています。アプリの画面や動作を組み立てるために必要です。
97	<code>: '\$prefix\$generatedText';</code>	Dart の1つの命令をここで終えています。変数宣言や処理の区切りとして必要です。
98	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
99	<code>if (_hasSentenceEnd(message)) {</code>	条件が当てはまる時だけ中の処理を実行します。入力チェックや分岐に使います。
100	<code>return message;</code>	整えた文字列を呼び出し元へ返します。画面のメッセージ欄に入れるためです。
101	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
102	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
103	<code>return '\$message。';</code>	整えた文字列を呼び出し元へ返します。画面のメッセージ欄に入れるためです。
104	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
105	<code>(空行)</code>	コードのまとまりを見やすく区切るための空行です。動作には直接影響しませんが、読みやすさを上げます。
106	<code>bool _hasSentenceEnd(String text) {</code>	変化する値を保存する変数です。画面の状態や処理に必要な情報を名前付きで持ちます。
107	<code>return text.endsWith('.') text.endsWith('!') text.endsWith('?');</code>	この関数やメソッドの結果を呼び出し元へ返します。後続の処理がこの値を使えます。
108	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。
109	<code>}</code>	ここまでが1つのブロックや部品設定の範囲です。閉じることで次の処理との境目を明確にします。